Spring Boot 4.0 Migration Guide

Andy Wilkinson edited this page last week · 61 revisions

This document is meant to help you migrate your application to Spring Boot 4.0.

Before You Start

Upgrade to the Latest `3.5.x` Version

Before you start the upgrade, make sure to upgrade to the latest available `3.5.x` version. This will make sure that you are building against the most recent dependencies of that line.

Please take time to review any calls you might be making to deprecated methods as these will be removed in Spring Boot 4.0.

Review Dependencies

The move to Spring Boot 4 will upgrade a number of dependencies and might require work on your end. You can review dependency management for `3.5.x` with dependency management for `4.0.x` to asses how your project is affected.

You may also use dependencies that are not managed by Spring Boot (e.g. Spring Cloud). As your project defines an explicit version for those, identify the compatible version before upgrading.

Review System Requirements

Spring Boot 4.0 requires Java 17 or later. Using the latest LTS release of Java is encouraged.

Spring Boot applications can also be developed using Kotlin where v2.2 or later must be used.

If you're using GraalVM's native-image, v25 or later must be used.

Spring Boot 4 is based on Jakarta EE 11 and requires a Servlet 6.1 baseline.

If you manage dependencies directly, make sure you update them accordingly. In particular, you must use Spring Framework 7.x.

Review Deprecations from Spring Boot 3.x

Classes, methods and properties that were deprecated in Spring Boot 3.x have been removed in this release. Please ensure that you aren't calling deprecated methods before upgrading.

Upgrade to Spring Boot 4

Once you have reviewed the state of your project and its dependencies, upgrade to the latest maintenance release of Spring Boot 4.0.

Features Removed From This Release

The features have been removed from this release and are no longer available

Undertow

Spring Boot 4 requires a Servlet 6.1 baseline, with which Undertow is not yet compatible. As a result, Undertow support is dropped, including the Undertow starter and the ability to use Undertow as an embedded server.

We do not recommend deploying Spring Boot 4.0 applications to a non-Servlet 6.1 compliant container.

Pulsar Reactive

Following the decision of removing reactor support in Spring Pulsar, Spring Boot no longer manages the reactive Pulsar client. Auto-configuration for Spring Pulsar Reactive has been removed as well.

Embedded Executable Uber Jar Launch Scripts

Support for embedded launch scripts, used to create "fully executable" jar files, has been removed. The support was specific to Unix-like operating systems and had a number of limitations, most notably conflicting with the recommendations for efficient deployments. If you still require similar functionality, we recommend looking at alternatives such as Gradle's application plugin.

You can still create uber jars with Spring Boot's build plugins and run them using `java -jar`.

Spring Session Hazelcast

Spring Session Hazelcast is now under the leadership of the Hazelcast team. As such, direct support for Spring Session Hazelcast has been removed from Spring Boot itself.

Spring Session MongoDB

Spring Session MongoDB is now under the leadership of the MongoDB team. As such, direct support for Spring Session MongoDB has been removed from Spring Boot itself.

Spock Integration

Spring Boot's Spock integration has been removed as Spock does not yet support Groovy 5.

Module Dependencies

Spring Boot 4.0 has a new modular design and now ships smaller focused modules rather than several large jars. As a result, this release has a more consistent set of "starter" POMs, and features that only relied on a third-party dependency to work can require an additional starter.

You will need to make dependency modifications if your current application doesn't use Spring Boot "starter" POMs.

Main Code

The new modules and "starter" POMs follow a convention that lets you identify where the support for a given technology comes from:

- All Spring Boot modules are named `spring-boot-<technology>`.
- The root package of each module is `org.springframework.boot.<technology>`.
- All "starter" POMs are named `spring-boot-starter-<technology>`.

To illustrate the above, support for GraphQL is in `spring-boot-graphql` with a root package of `org.springframework.boot.graphql`. The "starter" POM is `spring-boot-starter-graphql`.

Note	Review the list of "starter" POMs below based on the technologies that you are using and review your dependencies. You may face issue if you use a technology that previously had no "starter" POM. For instance, if you are using Flyway or Liquibase you used to only have the relevant third-party dependency. You now need to replace that with `spring-boot-starter-flyway` or `spring-boot-starter-liquibase`, respectively.

Test Code

The modularity also applies to the test infrastructure. A similar convention is applied:

- All Spring Boot test modules are named `spring-boot-<technology>-test`.
- The root package of each such module is `org.springframework.boot.<technology>.test`.
- All test "starter" POMs are named `spring-boot-starter-<technology>-test`.

To illustrate the above, test infrastructure for GraphQL is in `spring-boot-graphql-test` with a root package of `org.springframework.boot.graphql.test`. The "starter" POM is `spring-boot-starter-graphql-test`.

Note	If you are using a technology that defines its own test infrastructure, you should swap it to the `spring-boot-starter-<technology>-test` equivalent. In the example above, `spring-boot-graphql-starter-test` has a dependency on `org.springframework.graphql:spring-graphql-test`. Not doing so could lead to issues that are hard to track. For example, `@WithMockUser` and `@WithUserDetails` from `spring-security-test` now requires `spring-boot-starter-security-test` to operate properly.

Tip	Given that all test "starter" POMs bring `spring-boot-starter-test` transitively, you don't need to define this starter anymore. Rather, you should list the starters of the technologies that are under test for the module.

Starters

This release harmonizes the use of starters in two ways:

- Most technologies covered by Spring Boot have a dedicated starter.
- Each starter has a test starter companion.

Reviewing your test dependencies is important. The following table shows the "starter" POM you should be using to support a specific technology:

Technology	Main Dependency	Test Dependency
Core Starters		
AspectJ	`spring-boot-starter-aspectj`	`spring-boot-starter-aspectj-test`
Cloud Foundry Support	`spring-boot-starter-cloudfoundry`	`spring-boot-starter-cloudfoundry-test`
Jakarta Validation	`spring-boot-starter-validation`	`spring-boot-starter-validation-test`
Kotlinx Serialization JSON	`spring-boot-starter-kotlinx-serialization-json`	`spring-boot-starter-kotlinx-serialization-json-test`
Web Server Starters		
Jetty	`spring-boot-starter-jetty`	*none*
Reactor Netty	`spring-boot-starter-reactor-netty`	*none*
Tomcat	`spring-boot-starter-tomcat`	*none*
Web Client Starters		
Spring's Imperative `RestClient` and `RestTemplate`	`spring-boot-starter-restclient`	`spring-boot-starter-restclient-test`
Spring's Reactive `WebClient`	`spring-boot-starter-webclient`	`spring-boot-starter-webclient-test`
Web Starters		
Jersey	`spring-boot-starter-jersey`	`spring-boot-starter-jersey-test`
Spring GraphQL	`spring-boot-starter-graphql`	`spring-boot-starter-graphql-test`
Spring HATEOAS	`spring-boot-starter-hateoas`	`spring-boot-starter-hateoas-test`
Spring REST Docs		`spring-boot-starter-restdocs` (since Spring Boot 4.0.4)
Spring Session Data Redis	`spring-boot-starter-session-data-redis`	`spring-boot-starter-session-data-redis-test`
Spring Session JDBC	`spring-boot-starter-session-jdbc`	`spring-boot-starter-session-jdbc-test`
Spring Web MVC	`spring-boot-starter-webmvc`	`spring-boot-starter-webmvc-test`
Spring WebFlux	`spring-boot-starter-webflux`	`spring-boot-starter-webflux-test`
Spring Webservices	`spring-boot-starter-webservices`	`spring-boot-starter-webservices-test`
Database Starters		
Cassandra	`spring-boot-starter-cassandra`	`spring-boot-starter-cassandra-test`
Couchbase	`spring-boot-starter-couchbase`	`spring-boot-starter-couchbase-test`
Elasticsearch	`spring-boot-starter-elasticsearch`	`spring-boot-starter-elasticsearch-test`
Flyway	`spring-boot-starter-flyway`	`spring-boot-starter-flyway-test`
JDBC	`spring-boot-starter-jdbc`	`spring-boot-starter-jdbc-test`
jOOQ	`spring-boot-starter-jooq`	`spring-boot-starter-jooq-test`
Liquibase	`spring-boot-starter-liquibase`	`spring-boot-starter-liquibase-test`
LDAP	`spring-boot-starter-ldap`	`spring-boot-starter-ldap-test`
MongoDB	`spring-boot-starter-mongodb`	`spring-boot-starter-mongodb-test`
Neo4J	`spring-boot-starter-neo4j`	`spring-boot-starter-neo4j-test`
R2DBC	`spring-boot-starter-r2dbc`	`spring-boot-starter-r2dbc-test`
Spring Data Starters		
Spring Data Cassandra	`spring-boot-starter-data-cassandra` or `spring-boot-starter-data-cassandra-reactive`	`spring-boot-starter-data-cassandra-test` or `spring-boot-starter-data-cassandra-reactive-test`
Spring Data Couchbase	`spring-boot-starter-data-couchbase` or `spring-boot-starter-data-couchbase-reactive`	`spring-boot-starter-data-couchbase-test` or `spring-boot-starter-data-couchbase-reactive-test`
Spring Data Elasticsearch	`spring-boot-starter-data-elasticsearch`	`spring-boot-starter-data-elasticsearch-test`
Spring Data JDBC	`spring-boot-starter-data-jdbc`	`spring-boot-starter-data-jdbc-test`
Spring Data JPA (using Hibernate)	`spring-boot-starter-data-jpa`	`spring-boot-starter-data-jpa-test`
Spring Data LDAP	`spring-boot-starter-data-ldap`	`spring-boot-starter-data-ldap-test`
Spring Data MongoDB	`spring-boot-starter-data-mongodb` or `spring-boot-starter-data-mongodb-reactive`	`spring-boot-starter-data-mongodb-test` or `spring-boot-starter-data-mongodb-reactive-test`
Spring Data Neo4J	`spring-boot-starter-data-neo4j`	`spring-boot-starter-data-neo4j-test`
Spring Data R2DBC	`spring-boot-starter-data-r2dbc`	`spring-boot-starter-data-r2dbc-test`
Spring Data Redis	`spring-boot-starter-data-redis` or `spring-boot-starter-data-redis-reactive`	`spring-boot-starter-data-redis-test` or `spring-boot-starter-data-redis-reactive-test`
Spring Data REST	`spring-boot-starter-data-rest`	`spring-boot-starter-data-rest-test`
IO Starters		
Hazelcast	`spring-boot-starter-hazelcast`	`spring-boot-starter-hazelcast-test`
Mail	`spring-boot-starter-mail`	`spring-boot-starter-mail-test`
Quartz	`spring-boot-starter-quartz`	`spring-boot-starter-quartz-test`
SendGrid	`spring-boot-starter-sendgrid`	`spring-boot-starter-sendgrid-test`
Spring Caching Support	`spring-boot-starter-cache`	`spring-boot-starter-cache-test`
Spring Batch (with JDBC)	`spring-boot-starter-batch-jdbc`	`spring-boot-starter-batch-jdbc-test`
Spring Batch (without JDBC)	`spring-boot-starter-batch`	`spring-boot-starter-batch-test`
JSON Starters		
GSON	`spring-boot-starter-gson`	`spring-boot-starter-gson-test`
Jackson	`spring-boot-starter-jackson`	`spring-boot-starter-jackson-test`
JSONB	`spring-boot-starter-jsonb`	`spring-boot-starter-jsonb-test`
Messaging Starters		
ActiveMQ	`spring-boot-starter-activemq`	`spring-boot-starter-activemq-test`
Artemis	`spring-boot-starter-artemis`	`spring-boot-starter-artemis-test`
JMS	`spring-boot-starter-jms`	`spring-boot-starter-jms-test`
RSocket	`spring-boot-starter-rsocket`	`spring-boot-starter-rsocket-test`
Spring AMQP	`spring-boot-starter-amqp`	`spring-boot-starter-amqp-test`
Spring Integration	`spring-boot-starter-integration`	`spring-boot-starter-integration-test`
Spring for Apache Kafka	`spring-boot-starter-kafka`	`spring-boot-starter-kafka-test`
Spring for Apache Pulsar	`spring-boot-starter-pulsar`	`spring-boot-starter-pulsar-test`
Websockets	`spring-boot-starter-websocket`	`spring-boot-starter-websocket-test`
Security Starters		
Spring Security	`spring-boot-starter-security`	`spring-boot-starter-security-test`
Spring Security OAuth Authorization Server	`spring-boot-starter-security-oauth2-authorization-server`	`spring-boot-starter-security-oauth2-authorization-server-test`
Spring Security OAuth Client	`spring-boot-starter-security-oauth2-client`	`spring-boot-starter-security-oauth2-client-test`
Spring Security OAuth Resource Server	`spring-boot-starter-security-oauth2-resource-server`	`spring-boot-starter-security-oauth2-resource-server-test`
Spring Security SAML	`spring-boot-starter-security-saml2`	`spring-boot-starter-security-saml2-test`
Templating Starters		
Freemarker	`spring-boot-starter-freemarker`	`spring-boot-starter-freemarker-test`
Groovy Templates	`spring-boot-starter-groovy-templates`	`spring-boot-starter-groovy-templates-test`
Mustache	`spring-boot-starter-mustache`	`spring-boot-starter-mustache-test`
Thymeleaf	`spring-boot-starter-thymeleaf`	`spring-boot-starter-thymeleaf-test`
Production-Ready Starters		
Actuator	`spring-boot-starter-actuator`	`spring-boot-starter-actuator-test`
Micrometer Metrics	`spring-boot-starter-micrometer-metrics`	`spring-boot-starter-micrometer-metrics-test`
OpenTelemetry	`spring-boot-starter-opentelemetry`	`spring-boot-starter-opentelemetry-test`
Zipkin	`spring-boot-starter-zipkin`	`spring-boot-starter-zipkin-test`

Modules

If you prefer to not use "starter" POMs, you can instead declare direct module dependencies:

Technology	Main Dependency	Test Dependency
Core Modules		
Cloud Foundry Support	`spring-boot-cloudfoundry`	*none*
Jakarta Validation	`spring-boot-validation`	*none*
Kotlin Serialization	`spring-boot-kotlin-serialization`	*none*
Reactor	`spring-boot-reactor`	*none*
Web Server Modules		
Jetty	`spring-boot-jetty`	*none*
Reactor Netty	`spring-boot-reactor-netty`	*none*
Tomcat	`spring-boot-tomcat`	*none*
Web Client Modules		
Spring's Imperative `RestClient` and `RestTemplate`	`spring-boot-restclient`	`spring-boot-restclient-test`
Spring's Reactive `WebClient`	`spring-boot-webclient`	`spring-boot-webclient-test`
Web Modules		
Jersey	`spring-boot-jersey`	*none*
Spring GraphQL	`spring-boot-graphql`	`spring-boot-graphql-test`
Spring HATEOAS	`spring-boot-hateoas`	*none*
Spring REST Docs		`spring-boot-restdocs`
Spring Session Data Redis	`spring-boot-session-data-redis`	*none*
Spring Session JDBC	`spring-boot-session-jdbc`	*none*
Spring Web MVC	`spring-boot-webmvc`	`spring-boot-webmvc-test`
Spring WebFlux	`spring-boot-webflux`	`spring-boot-webflux-test`
Spring Webservices	`spring-boot-webservices`	`spring-boot-webservices-test`
Database Modules		
Cassandra	`spring-boot-cassandra`	*none*
Couchbase	`spring-boot-couchbase`	*none*
Elasticsearch	`spring-boot-elasticsearch`	*none*
Flyway	`spring-boot-flyway`	*none*
JDBC	`spring-boot-jdbc`	`spring-boot-jdbc-test`
jOOQ	`spring-boot-jooq`	`spring-boot-jooq-test`
Liquibase	`spring-boot-liquibase`	*none*
LDAP	`spring-boot-ldap`	*none*
MongoDB	`spring-boot-mongodb`	*none*
Neo4J	`spring-boot-neo4j`	*none*
R2DBC	`spring-boot-r2dbc`	*none*
Spring Data Modules		
Spring Data Cassandra	`spring-boot-data-cassandra`	`spring-boot-data-cassandra-test`
Spring Data Couchbase	`spring-boot-data-couchbase`	`spring-boot-data-couchbase-test`
Spring Data Elasticsearch	`spring-boot-data-elasticsearch`	`spring-boot-data-elasticsearch-test`
Spring Data JDBC	`spring-boot-data-jdbc`	`spring-boot-data-jdbc-test`
Spring Data JPA (using Hibernate)	`spring-boot-data-jpa`	`spring-boot-data-jpa-test`
Spring Data LDAP	`spring-boot-data-ldap`	`spring-boot-data-ldap-test`
Spring Data MongoDB	`spring-boot-data-mongodb`	`spring-boot-data-mongodb-test`
Spring Data Neo4J	`spring-boot-data-neo4j`	`spring-boot-data-neo4j-test`
Spring Data R2DBC	`spring-boot-data-r2dbc`	`spring-boot-data-r2dbc-test`
Spring Data Redis	`spring-boot-data-redis`	`spring-boot-data-redis-test`
Spring Data REST	`spring-boot-data-rest`	*none*
IO Modules		
Hazelcast	`spring-boot-hazelcast`	*none*
Mail	`spring-boot-mail`	*none*
Quartz	`spring-boot-quartz`	*none*
SendGrid	`spring-boot-sendgrid`	*none*
Spring Caching Support	`spring-boot-cache`	`spring-boot-cache-test`
Spring Batch (with JDBC)	`spring-boot-batch-jdbc`	*none*
Spring Batch (without JDBC)	`spring-boot-batch`	*none*
JSON Modules		
GSON	`spring-boot-gson`	*none*
Jackson	`spring-boot-jackson`	*none*
JSONB	`spring-boot-jsonb`	*none*
Messaging Modules		
ActiveMQ	`spring-boot-activemq`	*none*
Artemis	`spring-boot-artemis`	*none*
JMS	`spring-boot-jms`	*none*
RSocket	`spring-boot-rsocket`	`spring-boot-rsocket-test`
Spring AMQP	`spring-boot-amqp`	*none*
Spring Integration	`spring-boot-integration`	*none*
Spring for Apache Kafka	`spring-boot-kafka`	*none*
Spring for Apache Pulsar	`spring-boot-pulsar`	*none*
Websockets	`spring-boot-websocket`	*none*
Security Modules		
Spring Security	`spring-boot-security`	`spring-boot-security-test`
Spring Security OAuth Authorization Server	`spring-boot-security-oauth2-authorization-server`	*none*
Spring Security OAuth Client	`spring-boot-security-oauth2-client`	*none*
Spring Security OAuth Resource Server	`spring-boot-security-oauth2-resource-server`	*none*
Spring Security SAML	`spring-boot-security-saml2`	*none*
Templating Modules		
Freemarker	`spring-boot-freemarker`	*none*
Groovy Templates	`spring-boot-groovy-templates`	*none*
Mustache	`spring-boot-mustache`	*none*
Thymeleaf	`spring-boot-thymeleaf`	*none*
Production-Ready Modules		
Health	`spring-boot-health`	*none*
Micrometer Metrics	`spring-boot-micrometer-metrics`	`spring-boot-micrometer-metrics-test`
Micrometer Observation	`spring-boot-micrometer-observation`	*none*
Micrometer Tracing	`spring-boot-micrometer-tracing`	`spring-boot-micrometer-tracing-test`
Micrometer Tracing Brave	`spring-boot-micrometer-tracing-brave`	*none*
Micrometer Tracing OpenTelemetry	`spring-boot-micrometer-tracing-opentelemetry`	*none*
OpenTelemetry	`spring-boot-opentelemetry`	*none*
Zipkin	`spring-boot-zipkin`	*none*

Classic Starters

If you're upgrading an existing application and just want to get running quickly, you can use "Classic Starter POMs". The classic starters provide all modules, but exclude all of their transitive dependencies.

This provides a setup very similar to the previous generation of Spring Boot where all auto-configuration classes are available.

To use the classic starters, update your build as follows:

Previous Starter	Classic Equivalent
`spring-boot-starter`	`spring-boot-starter-classic`
`spring-boot-starter-test`	`spring-boot-starter-test-classic`

Note	We recommend that you eventually migrate your application away from using the classic starters.

Deprecated Starters

Serveral starter POMs have been renamed to provide better alignment with their corresponding module. The older starters remain, but should be considered deprecated and will be removed in a future release.

You should update your POM as follows:

Deprecated Starter	Replacement
`spring-boot-starter-oauth2-authorization-server`	`spring-boot-starter-security-oauth2-authorization-server`
`spring-boot-starter-oauth2-client`	`spring-boot-starter-security-oauth2-client`
`spring-boot-starter-oauth2-resource-server`	`spring-boot-starter-security-oauth2-resource-server`
`spring-boot-starter-web`	`spring-boot-starter-webmvc`
`spring-boot-starter-web-services`	`spring-boot-starter-webservices`

Package Organization

The modularization also has an impact on the package structure of the project. Each module now starts with a dedicated `org.springframework.boot.<module>` package. Depending on the scope of the module, it can contain APIs, auto-configurations, actuator-related support, etc.

Considerations For Projects Building Their Own Starters

Due to the modularization effort, supporting both Spring Boot 3 and Spring Boot 4 within the same artifact is strongly discouraged.

Migration Strategy

Depending on the size of your application, you may want to adapt directly to the new modularization or perform the upgrade in two steps.

Adding `spring-boot-starter-classic` (and `spring-boot-starter-test-classic` for tests) gets you back to a classpath where all the infrastructure is available. You can use this intermediary state to fix broken imports and validate your application works.

Once the migration is complete, you can remove the classic starters and use the updated imports to identify the missing "starter" POMs.

Review Other Release Notes

This major release moves to major releases of the rest of the Spring portfolio. As such, make sure to review the release notes of other projects your application is using:

- Spring AMQP 4.0
- Spring Batch 6.0
- Spring Data 2025.1
- Spring GraphQL 2.0
- Spring Framework 7.0
- Spring Integration 7.0
- Spring for Apache Kafka 4.0

Pages 256

Pages
Home
Supported Versions

Release Notes
v4.1 (preview)
v4.0
v3.5
v3.4
v3.3
Older Versions

Migration Guides
v3.5 → v4.0
v2.7 → v3.0
v2.4+ Config Data
v1.5 → v2.0

Help
How to get help
Configuration Binding
IDE Binding Features
Building on Spring Boot
Spring Boot with GraalVM
Building OCI Images With Spring Boot

Development Process
Antora Documentation
Build Status
Working with the Code
Team Practices
Working with Git Branches
Merging Pull Requests
Useful Build Commands
Useful Git Aliases
GitHub Actions
GitHub Issues
Maven POM Files
Performance Tuning
Generating SSL KeyStores
Deprecations
Creating a New Maintenance Branch
Restructuring

Clone this wiki locally
https://github.com/spring-projec

- Spring for Apache Pulsar 2.0
- Spring Security 7.0
- Spring REST Docs 4.0
- Spring Session 4.0
- Spring WS 5.0

Upgrading Core Features

JSpecify Nullability annotations

Spring Boot 4.0 adds JSpecify nullability annotations. If you are using a null checker in your build or using Kotlin, this could lead to compilation failures because of now nullable or non-nullable types.

In addition, if you previously used Spring Framework's nullable annotations from the `org.springframework.lang` package you should review "Migrating from Spring null-safety annotations".

Logback Default Charset

The default Charset for Logback has been harmonized with the behavior of Log4j2. By default, the charset of a log file is `UTF-8`. For console log, we use the `Console#charset()` if it is available, otherwise we use `UTF-8`.

BootstrapRegistry and EnvironmentPostProcessor package changes

The `BootstrapRegistry` and related classes have moved from `org.springframework.boot` to `org.springframework.boot.bootstrap`. The `EnvironmentPostProcessor` interface has also moved from `org.springframework.boot.env` to `org.springframework.boot`.

If you have deep integrations with Spring Boot you may need to update both your code and your `spring.factories` files.

Note	The deprecated form of the `EnvironmentPostProcessor` is still available in Spring Boot 4.0, but will be removed at a later date.

Property Mapper API Changes

The `PropertyMapper` class no longer calls adapter or predicate methods by default when the source value is `null`. This has removed the need for the `alwaysApplyingNotNull()` method which has been removed.

If you need to perform a mapping even for `null` values you can use the new `always()` method.

For example,

```
map.from(source::method).to(destination::method);
```

Will not call `destination.method(…)` if `source.method()` returns `null`.

Where as:

```
map.from(source::method).always().to(destination::method);
```

Will call `destination.method(null)` if `source.method()` returns `null`.

If you use the `PropertyMapper`, you might want to review commit `239f384ac0` which shows how Spring Boot itself adapted to the new API.

DevTools Live Reload Support

DevTools Live Reload support is now disabled by default. If you want to use live reload please set `spring.devtools.livereload.enabled` to `true`.

Upgrading Dependencies and Build Plugins

Optional Dependencies in Maven

Optional dependencies are no longer included in uber jars. If you need them, you can use the configuration setting `<includeOptional>true</includeOptional>`.

AOP Starter POM

To clarify the scope of `spring-boot-starter-aop`, it has been renamed to `spring-boot-starter-aspectj`. If you have added the starter explicitly in your application, please review if you actually need it before using the replacement.

To figure this out, review the following:

- Your application does not use AspectJ, typically an annotation in the `org.aspectj.lang.annotation` package.
- Search for usage of `org.aspectj.lang.annotation.Aspect` on the classpath of your application. They usually document the annotation that triggers the feature, such as Micrometer's `@Timed` or `@Counted`.

Dependency Management for Spring Retry

With the portfolio moving from Spring Retry to new core features of Spring Framework, dependency management for Spring Retry has been removed.

If your application still relies on Spring Retry, an explicit version is now required. Please consider moving your use of Spring Retry to Spring Framework.

Dependency Management for Spring Authorization Server

Spring Authorization Server is now part of Spring Security. Explicit dependency management has been removed in favor of what's already provided by Spring Security.

As a result, you can no longer override the version of Spring Authorization Server using the `spring-authorization-server.version` property. If you need to do this going forward, use `spring-security.version`.

Classic Uber-Jar Loader Support

The classic uber-jar loader has been removed from this release. You should remove any loader implementation configuration from your build file.

For maven this means removing the following:

```
<loaderImplementation>CLASSIC</loaderImplementation>
```

For Gradle, it will be lines similar to:

```
loaderImplementation = org.springframework.boot.loader.tools.LoaderImplementation.CLASSIC
```

Cyclone DX Gradle Plugin

The minimum supported version of the CycloneDX Gradle plugin is now 3.0.0.

Upgrading Jackson

Spring Boot now uses Jackson 3 as its preferred JSON library. Jackson 3 uses new group IDs and package names with `com.fasterxml.jackson` becoming `tools.jackson`. An exception to this is the `jackson-annotations` module which continues to use the `com.fasterxml.jackson.core` group ID and `com.fasterxml.jackson.annotation` package. To learn more about the changes in Jackson 3, refer to the Jackson wiki.

For libraries that require Jackson 2, dependency management for Jackson 2 remains and a Jackson 2 `ObjectMapper` can be used alongside Boot's auto-configuration for Jackson 3 if needed.

A number of classes have been renamed for consistency with Jackson 3:

- `JsonObjectSerializer` to `ObjectValueSerializer`.
- `JsonValueDeserializer` to `ObjectValueDeserializer`.
- `Jackson2ObjectMapperBuilderCustomizer` to `JsonMapperBuilderCustomizer`.

`@JsonComponent`, `@JsonMixin`, and related supporting classes have been renamed to clarify that they are specific to Jackson and are not, necessarily, limited to JSON. `@JsonComponent` is now `@JacksonComponent` and `@JsonMixin` is now `@JacksonMixin`. Supporting classes have been renamed in the same manner replacing `Json` with `Jackson`.

The JSON-specific `spring.jackson.read.*` and `spring.jackson.write.*` properties have moved beneath `spring.jackson.json.read` and `spring.jackson.json.write` respectively.

Where a Jackson 2 `JsonParser.Feature` has an equivalent Jackson 3 `JsonReadFeature`, `spring.jackson.parser.*` properties have been replaced with `spring.jackson.json.read` properties. Where a Jackson 2 `JsonParser.Feature` has a Jackson 3 equivalent that is not a `JsonReadFeature`, use a `JsonMapperBuilderCustomizer` to apply the desired configuration programatically.

As of Spring Boot 4, Jackson will detect all modules present on the classpath and will register them with the mapper instances. In Spring Boot 3, only "well-known" modules were registered automatically. You can disable this new behavior by setting the `spring.jackson.find-and-add-modules=false` property.

Jackson 2 Compatibility

To help with migrating from Jackson 2 to Jackson 3, the `spring.jackson.use-jackson2-defaults` property has been introduced. When set to `true`, the auto-configured `JsonMapper` will have defaults that align as closely as possible with those of Jackson 2 in Spring Boot 3.x.

We strongly recommend trying to adopt Jackson 3 if at all possible, however, if you are unable to do so Spring Boot 4.0 also provides a `spring-boot-jackson2` module. This module ships in a deprecated form and will be removed in a future release. It's intended as a stop-gap for users that need more time to migrate to Jackson 3.

To use the Jackson 2 module you can add the following dependency to your Maven POM:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-jackson2</artifactId>
</dependency>
```

Or if you use Gradle:

```
implementation("org.springframework.boot:spring-boot-jackson2")
```

Jackson 2 properties are available under `spring.jackson2`. These are equivalent to the `spring.jackson` properties provided by Spring Boot 3.5. Refer to the reference documentation for more information on using and configuring Jackson 2 with Spring Boot 4.

Upgrading Actuator and Production-Ready Features

Removed Support for javax.annotations.NonNull and org.springframework.lang.Nullable

Actuator endpoint parameters no longer can use `org.springframework.lang.Nullable` to declare that a parameter is optional. If you are using this annotation, you should migrate to `org.jspecify.annotations.Nullable`.

Liveness and Readiness Probes

The liveness and readiness probes are now enabled by default. This means the health endpoint now exposes the `liveness` and `readiness` groups by default.

If you do not need those probes, you can disable them by using the `management.endpoint.health.probes.enabled` property.

Upgrading Web Features

Fonts Added to Common Static Locations

If you use `PathRequest#toStaticResources`, please be advised that this release adds a location to `/fonts/**`. With no change on your end, this path will have the same security settings as what you had previously configured.

If you don't want that location to be included, you can exclude it like any other location:

```
pathRequest.toStaticResources().atCommonLocations().excluding(StaticResourceLocation.FONTS);
```

Spring Session

The properties for Spring Session Data Redis have been renamed to reflect the dependencies on Spring Data Redis. Properties that previously began with `spring.session.redis` now begin with `spring.session.data.redis`. Similarly, properties that previously began with `spring.session.mongodb` now begin with `spring.session.data.mongodb`.

HttpMessageConverters Deprecation

This release deprecates Spring Boot's `HttpMessageConverters` because Spring Framework improved the converter configuration in the traditional stack. `HttpMessageConverters` had several issues, including the fact that it conflates client and server converters.

If your application declares a custom `org.springframework.boot.http.converter.autoconfigure.HttpMessageConverters` bean, this is still supported but the type itself is deprecated. If you are contributing `HttpMessageConverter` beans to the context (like a `JacksonJsonHttpMessageConverter`), this is not supported anymore and you will need to update your configuration.

Instead, your application can declare one or more `ClientHttpMessageConvertersCustomizer` and `ServerHttpMessageConvertersCustomizer` that will let you customize converters in a flexible way. Each customizer can choose to contribute converters as "custom" converters considered before default ones, or instead to use a converter instance to replace a default converter that was auto-detected. You'll find more information and a sample in the reference documentation.

Jersey

Spring Boot 4.0 supports Jersey 4.0 which does not yet support Jackson 3. To process JSON with Jersey, use `spring-boot-jackson2` either in place of or alongside `spring-boot-jackson`.

Tomcat

If you're deploying a war file to Tomcat, you need to switch the dependency `spring-boot-starter-tomcat` to `spring-boot-starter-tomcat-runtime`.

Upgrading Data Features

Elasticsearch Client

In response to changes in Elasticsearch, auto-configuration for the now-deprecated low-level Elasticsearch `RestClient` has been replaced with auto-configuration for the new `Rest5Client`. If you were using Spring Boot's `RestClientBuilderCustomizer` to customize the client, you must now use `Rest5ClientBuilderCustomizer` instead.

As part of the changes in Elasticsearch, the client code has been consolidated in the `co.elastic.clients:elasticsearch-java` module, including built-in sniffer support. The `org.elasticsearch.client:elasticsearch-rest-client` and `org.elasticsearch.client:elasticsearch-rest-client-sniffer` modules are no longer required and Spring Boot's dependency management for them has been removed.

Support for the higher-level `ElasticsearchClient` and the Spring Data Elasticsearch-provided `ReactiveElasticsearchClient` remains. It has been updated to use the new low-level client.

Persistence Modules

A new `spring-boot-persistence` module has been created to house general persistence-related code and properties. Users of `@EntityScan` should adapt their imports to `org.springframework.boot.persistence.autoconfigure.EntityScan`.

The `spring.dao.exceptiontranslation.enabled` property is no longer supported. Please use `spring.persistence.exceptiontranslation.enabled` instead.

MongoDB

Some properties for configuring MongoDB have been updated so that their names reflect whether or not Spring Data MongoDB is required. Many properties whose names previously began with `spring.data.mongodb` now begin with `spring.mongodb`:

- `spring.mongodb.additional-hosts`
- `spring.mongodb.authentication-database`
- `spring.mongodb.database`
- `spring.mongodb.host`
- `spring.mongodb.password`
- `spring.mongodb.port`
- `spring.mongodb.protocol`
- `spring.mongodb.replica-set-name`
- `spring.mongodb.representation.uuid`
- `spring.mongodb.ssl.bundle`
- `spring.mongodb.ssl.enabled`
- `spring.mongodb.uri`
- `spring.mongodb.username`

Additionally, management-related properties have been renamed to use `mongodb` rather than `mongo` in their names:

- `management.health.mongodb.enabled`
- `management.metrics.mongodb.command.enabled`
- `management.metrics.mongodb.connectionpool.enabled`

The following properties that require Spring Data MongoDB are unchanged:

- `spring.data.mongodb.auto-index-creation`
- `spring.data.mongodb.field-naming-strategy`
- `spring.data.mongodb.gridfs.bucket`
- `spring.data.mongodb.gridfs.database`
- `spring.data.mongodb.repositories.type`

MongoDB UUID and BigDecimal Representations

Spring Data MongoDB no longer provide defaults for `UUID` and `BigInteger` / `BigDecimal` representations. This aligns with the driver recommendation to not favor a particular representation for `UUID` or `BigInteger` / `BigDecimal` to avoid representation changes caused by upgrades to a newer Spring Data version.

An explicit configuration is expected and the representations can be set using the `spring.mongodb.representation.uuid` and `spring.data.mongodb.representation.big-decimal` properties, respectively.

Hibernate Dependency Management

The dependency management for Hibernate has been harmonized to take relocations into account:

- `hibernate-jpamodelgen` is replaced by `hibernate-processor`.
- `hibernate-proxool` and `hibernate-vibur` are no longer published.

Upgrading Messaging Features

Kafka Streams Customization

Spring Boot's `StreamBuilderFactoryBeanCustomizer` has been removed in favor of Spring Kafka's `StreamsBuilderFactoryBeanConfigurer`. When migrating to the new configurer, be aware that it implements `Ordered` with a default value of `0`.

Spring Kafka Retry Features

Spring Kafka has moved its retry capabilities from Spring Retry to Spring Framework.

As a result, `spring.kafka.retry.topic.backoff.random` has been removed in favor of `spring.kafka.retry.topic.backoff.jitter`. The latter provides more flexibility over the former. See the documentation for more details.

Spring AMQP Retry Features

Spring AMQP has moved its retry capabilities from Spring Retry to Spring Framework.

Spring Boot offers a customization hook-point for retry features used by the `RetryTemplate` and message listeners. To make it more explicit, two dedicated customizers have been introduced: `RabbitTemplateRetrySettingsCustomizer`, and `RabbitListenerRetrySettingsCustomizer`.

If you were using `RabbitRetryTemplateCustomizer` to customize the retry settings according to a target, you will need to migrate to either of those interfaces.

Upgrading IO Features

Spring Batch

Spring Batch can now operate without a database (i.e. in memory), and the regular `spring-boot-starter-batch` uses this simplified mode.

On upgrade, Spring Batch will no longer store metadata in your existing database. You can either simplify your configuration and use this new mode, or restore the previous arrangement. To go back to using a database, you need to change to `spring-boot-starter-batch-jdbc`.

Upgrading Testing Features

Mockito Captor and Mock Annotations

The `MockitoTestExecutionListener` deprecated in Spring Boot 3.4 has been removed in this release. This might catch a few users out since the listener was only indirectly used and so deprecation warnings were easy to miss.

If you find your `@Mock` or `@Captor` annotated fields aren't working as expected, you should use `MockitoExtension` from Mockito itself.

Using MockMVC and @SpringBootTest

Using the `@SpringBootTest` annotation will no longer provide any MockMVC support. If you want to use MockMVC in your tests you should now add an `@AutoConfigureMockMvc` annotation to the test class.

In addition, HtmlUnit specific settings have now moved under an `htmlUnit` attribute. For example, in Spring Boot 3.5 you may have used `@AutoConfigureMockMvc(webClientEnabled=false, webDriverEnabled=false)` to disable HtmlUnit features. In Spring Boot 4.0 you'd use `@AutoConfigureMockMvc(htmlUnit = @HtmlUnit(webClient = false, webDriver = false))`.

Using WebClient or TestRestTemplate and @SpringBootTest

Using the `@SpringBootTest` annotation will no longer provide any `WebClient` or `TestRestTemplate` beans. If you want to use a `TestRestTemplate` you should add an `@AutoConfigureTestRestTemplate` annotation to the test class. Dependencies on `org.springframework.boot:spring-boot-resttestclient` and `org.springframework.boot:spring-boot-restclient` are also required.

In addition, you might want to consider replacing any use of `TestRestTemplate` with the new `RestTestClient` class. To configure this, add an `@AutoConfigureRestTestClient` annotation to the test class.

TestRestTemplate Failures

In the event of a compilation failure in your tests for `TestRestTemplate`, add a test-scoped dependency on `org.springframework.boot:spring-boot-resttestclient`. The package holding that class will also need to be updated to `org.springframework.boot.resttestclient.TestRestTemplate`.

A runtime dependency on `org.springframework.boot:spring-boot-restclient` is also required.

@PropertyMapping Annotation

The `@PropertyMapping` annotation has been relocated from the `org.springframework.boot.test.autoconfigure.properties` package to `org.springframework.boot.test.context`. In addition, the `skip` attribute now takes `org.springframework.boot.test.context.PropertyMapping.Skip` rather than `org.springframework.boot.test.autoconfigure.properties.Skip`.

`@MockBean` and `@SpyBean` deprecations

Spring Boot's `@MockBean` and `@SpyBean` support has been deprecated in this release, in favor of `@MockitoBean` and `@MockitoSpyBean` support. You can temporarily work around this deprecation by adding `@SuppressWarnings("removal")` in your tests, or using a custom annotation if there are many occurrences in your codebase.

While working around the deprecation can be useful, it is only a temporary situation as `@MockBean` and `@SpyBean` support will be removed in the future. If your tests are using `@MockBean` and `@SpyBean` as fields in test classes, you can consider a direct replacement:

```
@SpringBootTest
class ApplicationTests {

    @MockitoBean
    private GreetingService greetingService;

    @Test
    void check() {
        // ...
    }
}
```

Behavior is explained in more details in the Spring Framework reference docs section named `@MockitoBean` and `@MockitoSpyBean`.

There is a key difference between `@MockBean` / `@SpyBean` and `@MockitoBean` / `@MockitoSpyBean`. The new annotations are allowed to be used as fields in test classes, but not in `@Configuration` classes. Using those in `@Configuration` is a way to declare a set of mocked beans without repeating those fields in different test classes:

```
@SpringBootTest
@Import(TestConfig.class)
class ApplicationTests {

    @Test
    void check() {
        // ...
    }
}

@TestConfiguration
public class TestConfig {
    @MockBean
    private UserService userService;

    @MockBean
    private OrderService orderService;

    @MockBean
    private PrintingService ps1;
}
```

Instead, those "shared mocked beans" can be declared directly on the test class like this:

```
@SpringBootTest
@MockitoBean(types = {OrderService.class, UserService.class})
@MockitoBean(name = "ps1", types = PrintingService.class)
class ApplicationTests {

    @Test
    void check() {
        // ...
    }
}
```

If declaring those on the test class itself (or any super-class in its hierarchy) is not practical, you can also consider a custom annotation:

```
@SpringBootTest
@SharedMocks
class ApplicationTests {

    @Test
    void check() {
        // ...
    }
}

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@MockitoBean(types = {OrderService.class, UserService.class})
@MockitoBean(name = "ps1", types = PrintingService.class)
public @interface SharedMocks {
}
```